THE COMPLETE GUIDEBOOK TO ZERO-KNOWLEDGE BLOCKCHAIN & DAPP ENGINEERING

From Distributed Systems First Principles to Production Deployment on Midnight Network with WSL2, Ubuntu, Docker & Midnight Lace Wallet

---

Project: KNIGHT.VAULT Protocol	**Network:** Midnight Blockchain (Preprod / Preview)
Smart Contracts: Compact Circuits	**ZK Prover:** Local Docker ProofServer (:6300) / Railway
Wallet System: Midnight Lace DApp Connector (0 Gas)	**Architect:** Kshitij Adakane (Systems Builder)

Chapter 1: The First Principles of Computing & Blockchains

To understand what a decentralized application (DApp) is, we must first break down the concept of a **ledger** from its most elementary mathematical roots.

1.1 What is a Ledger?

At its core, a ledger is an append-only chronological record of facts and balance transitions. Historically, ledgers were centralized: a trusted entity (bank, registry, or database administrator) held the master copy. If the central authority fails, gets censored, or makes an error, the entire system state becomes corrupt.

1.2 What is a Blockchain?

A blockchain is a **decentralized, cryptographically-secured distributed ledger** maintained by a peer-to-peer (P2P) network of independent nodes without requiring a central coordinator.

- **Blocks:** Batches of valid transactions bundled with timestamp, nonce, and root hashes.
- **Cryptographic Hash Linking:** Every block contains the hash of the preceding block header, making state immutable.
- **Consensus:** Algorithms (AURA, GRANDPA, PoS) enabling global agreement on the valid state order.
- **Smart Contracts:** Deterministic state machine programs that run autonomously on-chain.

Chapter 2: Production Development Environment Setup (WSL2, Docker & Tools)

Building on Midnight Network requires compiling Compact smart contracts, running local Zero-Knowledge proving engines, and orchestrating client SDKs. Below are the precise steps and real-world hurdles encountered when configuring developer workstations.

2.1 Windows Subsystem for Linux (WSL2) & Ubuntu Setup

Because the Compact compiler toolchain and cryptographic ZK-SNARK provers are native Linux binaries, Windows developers must execute their build environments inside **WSL2 (Ubuntu 22.04 LTS or 24.04 LTS)**.

1. Open Windows PowerShell as Administrator and enable WSL2:

```
wsl --install -d Ubuntu-22.04
wsl --set-default-version 2
```

2. **Crucial Memory Tuning (.wslconfig):** ZK-SNARK circuit proving requires substantial memory. Create or edit C:\Users\<YourUser>\.wslconfig to allocate adequate RAM to prevent Out-Of-Memory (OOM) crashes:

```
[wsl2]
memory=12GB
swap=4GB
localhostForwarding=true
```

3. Inside Ubuntu, install essential system build dependencies:

```
sudo apt update && sudo apt install -y curl git build-essential pkg-config libssl-dev
```

2.2 Docker Desktop & ProofStation Container Setup

The Midnight ProofServer is responsible for compiling Halo2 / Plonk zero-knowledge circuits into mathematical proof transcripts on port **6300**.

- **WSL Integration:** Open Docker Desktop → *Settings* → *Resources* → *WSL Integration* → enable **Ubuntu-22.04**.
- **Starting the ProofServer:** Run the official Midnight ProofServer container:

```
docker run -d --name proof-server -p 6300:6300 midnightnetwork/proof-server:latest
```

- **Verify Container Health:** Confirm the server is responding on localhost:

```
curl http://localhost:6300/health
# Response: {"status":"ok","service":"proof-server"}
```

> ***Troubleshooting Port 6300:*** *If Docker reports 'port already in use', find the conflicting process with netstat -ano | findstr :6300 or kill dangling proof containers with docker rm -f proof-server.*

Chapter 3: Midnight Lace Wallet Integration & Sponsored Dust Architecture

Midnight Network introduces a dual-token paradigm: **tNIGHT** (unshielded/shielded settlement currency) and **DUST** (non-transferable computational resource used for gas fees).

3.1 Midnight Lace Browser Wallet Configuration

1. Install the **Midnight Lace Wallet Extension** from the Midnight Developer Portal into Google Chrome, Brave, or Edge.

2. Switch the active network to **Midnight Preprod** with the official GraphQL indexer URL: https://indexer.preprod.midnight.network/api/v4/graphql.
3. Fund your address via the **Midnight Testnet Faucet** to receive initial tNIGHT.
4. **Handshake Authorization:** In TypeScript, the frontend connects to Lace using:

```
const midnightApi = window.midnight?.mnLace;
if (!midnightApi) throw new Error('Midnight Lace wallet not detected');
const wallet = await midnightApi.enable();
const state = await wallet.state();
```

3.2 Zero-Gas Dust Balancing Flow

Traditional blockchains require every payer to hold gas tokens. In **KNIGHT.VAULT**, transactions are balanced via balanceUnsealedTransaction through Lace's ProofStation relay, allowing users to interact with **0.00 NIGHT user gas**.

Chapter 4: Compact Smart Contract Design & ZK Proofs

The smart contract is written in **Compact**, Midnight's domain-specific language for zero-knowledge smart contracts.

```
// payment.compact (Midnight stdlib)
contract PaymentVault {
  ledger balance: Counter;
  ledger totalDeposited: Counter;
  ledger ownerCommitment: Bytes<32>;

  // Public Payer Inflow (Zero-Gas)
  export circuit receiveUnshielded(amount: Uint<64>): Void {
    balance.increment(amount);
    totalDeposited.increment(amount);
  }

  // Confidential Withdrawal (ZK Owner Auth)
  export circuit withdraw(
    amount: Uint<64>,
    recipient: Either<ZswapCoinPublicKey, ContractAddress>
  ): Void {
    witness ownerSecretKey: Bytes<32>;
    assert persistentCommit(ownerSecretKey) == ownerCommitment;
    balance.decrement(amount);
    sendUnshielded(amount, recipient);
  }
}
```

4.1 How Privacy is Enforced Cryptographically

- **Private Witness (witness ownerSecretKey):** The secret key is evaluated strictly inside local browser memory. It is never broadcast over RPC or committed to the ledger.
- **Zero-Knowledge Commitment (persistentCommit):** The contract asserts that the private witness produces the published owner commitment without revealing the preimage.
- **Deterministic Ledger Counter:** The public vault balance increments and decrements transparently, while the withdrawer's identity remains completely shielded.

Chapter 5: Production Summary & Architect Credits

KNIGHT.VAULT represents an end-to-end realization of institutional privacy. By combining WSL2, Docker ProofServer, Compact Circuits, and the Midnight Lace Wallet, developers can build scalable, privacy-first DeFi applications.

Architect: *Kshitij Adakane • Vibe Coder, Systems Builder & Applied AI/ML*
GitHub: *https://github.com/KSHITIJadakane/KNIGHT.VAULT*